ESCUELA SUPERIOR POLITÉCNICA AGROPECUARIA DE MANABÍ

- **CARRERA:**

DESARROLLO DE SOFTWARE

- **ASIGNATURA:**

PROGRAMACION ORIENTADA A OBJETOS

- **ESTUDIANTE:**

CALDERON JARA GINA SARAHI

- **DOCENTE:**

ING. JALCA SALTOS LUIGGI

- **CURSO:**

SEGUNDO NIVEL PARALELO A

- **REPOSITORIO GITHUB:**

https://github.com/GinaCalderon/POO-2N-ESPAM-CJGS.git

2. Objetivo pedagógico

El presente informe tiene como objetivo aplicar los principios fundamentales de la Programación Orientada a Objetos (POO) en el diseño y desarrollo de un sistema de gestión para la producción y fileteo de atún. Mediante la creación de clases, constructores y métodos, se aplican conceptos como el encapsulamiento y las validaciones para garantizar que los datos y procesos cumplan con las reglas establecidas.

Asimismo, se busca demostrar que el sistema puede funcionar en dos lenguajes de programación, Java y Python, manteniendo la misma lógica y las reglas del proceso, aunque cada lenguaje utilice una sintaxis diferente.

3. Desarrollo del informe

3.1. Análisis del problema

El proceso de producción y fileteo de atún comprende varias etapas en las que se controla el peso de la materia prima y se registran las cantidades obtenidas durante el procesamiento. Para comprender su funcionamiento, se identificaron las siguientes etapas:





Método SCQA

El método SCQA permite identificar y organizar el problema mediante cuatro elemento, se utiliza para explicar las dificultades que pueden presentarse durante el proceso de fileteo y plantear una solución mediante un sistema desarrollado con Programación Orientada a Objetos.



3.2. Diagrama de clases UML

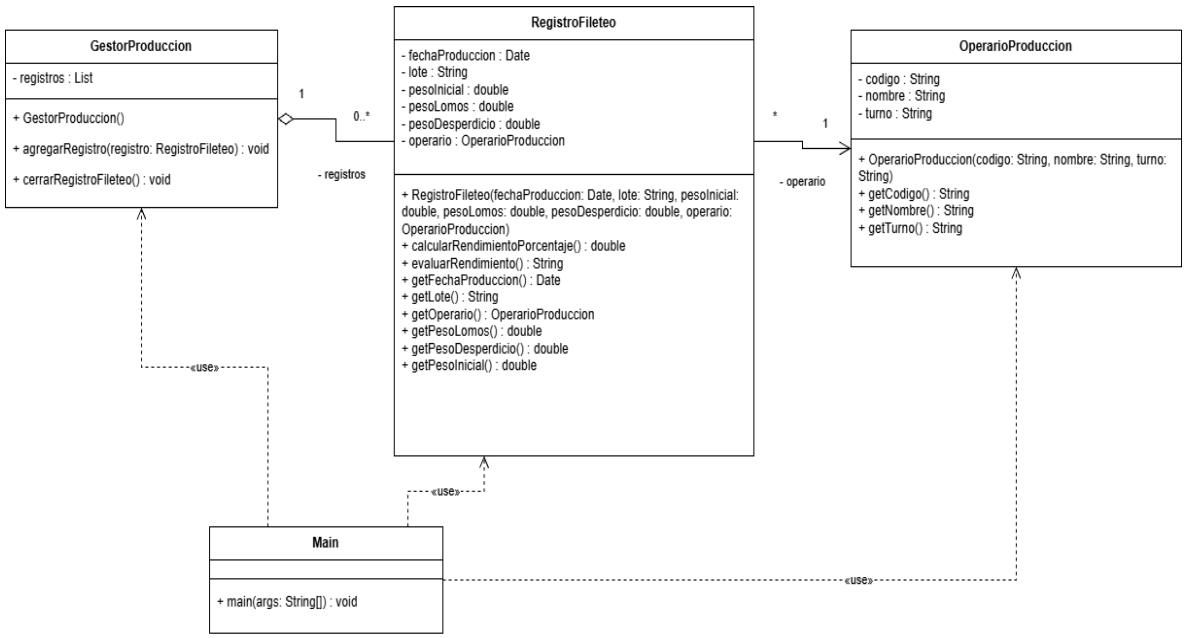
El diagrama representa la estructura del sistema de gestión de producción y fileteo de atún. En él se identifican las clases principales, sus atributos, métodos y las relaciones que permiten organizar el funcionamiento del sistema.

Clases:

- **OperarioProduccion:** representa al trabajador encargado del proceso de fileteo y almacena sus datos de identificación.
- **RegistroFileteo:** almacena la información de cada registro de producción, valida la conservación de la masa y calcula el rendimiento obtenido.
- **GestorProduccion:** administra los registros de fileteo y genera un resumen de los resultados de producción.

Elementos:

- **Atributos:** representan los datos que almacena cada clase.
- **Métodos:** representan las acciones que puede realizar cada clase.
- **Visibilidad:** el signo - indica atributos privados y el signo + indica métodos públicos.
- **Relaciones y multiplicidades:** muestran cómo se conectan las clases y cuántos objetos pueden participar en cada relación.



3.3. Desarrollo código Java

La clase de dominio más completo de nuestro sistema es RegistroFileteo.java, esta contiene validaciones, cálculos y reglas principales del proceso.

Desarrollo de la clase RegistroFileteo.java

→Atributos, constructor y validaciones:

- Encapsulamiento mediante atributos privados.
- Constructor parametrizado.
- Validación de conservación de masa.
- Validación de pesos.
- Inicialización de atributos con this.

03) Desarrollo de Código JAVA > SistemaProduccionFileteo > src > RegistroFileteo.java > RegistroFileteo > calcularRendimientoPorcentaje()

```java
import java.util.Date;
class RegistroFileteo
{
    private Date fechaProduccion;
    private String lote;
    private double pesoInicial;
    private double pesoLomos;
    private double pesoDesperdicio;
    private OperarioProduccion operario;

    //Constructor 1: recibe todos los datos, incluida la fecha

    public RegistroFileteo(Date fechaProduccion, String lote, double pesoInicial, double pesoLomos, double pesoDesperdicio, OperarioProduccion operario) {
        // Validación 1: Conservación de masa (pesoLomos + pesoDesperdicio == pesoInicial)
        // Se utiliza una tolerancia de 0.001 para controlar diferencias por precisión decimal.
        if (Math.abs((pesoLomos + pesoDesperdicio) - pesoInicial) > 0.001) {
            throw new IllegalArgumentException("Error de Invariante de Masa: pesoLomos + pesoDesperdicio debe ser igual al pesoInicial.");
        }

        if (pesoInicial <= 0 || pesoLomos < 0 || pesoDesperdicio < 0) {
            throw new IllegalArgumentException("Los pesos no pueden ser negativos y el peso inicial debe ser mayor a cero.");
        }

        //Inicialización de atributos

        this.fechaProduccion = fechaProduccion;
        this.lote = lote;
        this.pesoInicial = pesoInicial;
        this.pesoLomos = pesoLomos;
        this.pesoDesperdicio = pesoDesperdicio;
        this.operario = operario;
    }

    //Constructor 2: asigna automáticamente la fecha actual

    public RegistroFileteo(String lote, double pesoInicial,double pesoLomos, double pesoDesperdicio, OperarioProduccion operario) {
        this(new Date(), lote, pesoInicial, pesoLomos, pesoDesperdicio, operario);
    }
```

→Métodos y getters:

- Cálculo del porcentaje de rendimiento.
- Clasificación del resultado según los parámetros establecidos.
- Implementación de métodos públicos.
- Acceso controlado a los atributos mediante getters.

```java
    // Método para calcular el porcentaje de rendimiento de los lomos respecto al peso inicial
    public double calcularRendimientoPorcentaje() {
        return (pesoLomos / pesoInicial) * 100.0;
    }

    // Validación 2: Evaluar si el rendimiento es Bueno o Malo
    public String evaluarRendimiento() {
        double rendimiento = calcularRendimientoPorcentaje();
        // Si los lomos representan del 45% al 51% -> Rendimiento Bueno; de lo contrario -> Rendimiento Malo
        if (rendimiento >= 45.0 && rendimiento <= 51.0) {
            return "Rendimiento Bueno";
        } else {
            return "Rendimiento Malo";
        }
    }

// Getters para consultar los atributos
    public Date getFechaProduccion() {
        return fechaProduccion;
    }

    public String getLote() {
        return lote;
    }

    public OperarioProduccion getOperario() {
        return operario;
    }

    public double getPesoLomos() {
        return pesoLomos;
    }

    public double getPesoDesperdicio() {
        return pesoDesperdicio;
    }

    public double getPesoInicial() {
        return pesoInicial;
    }
}
```

Implementación del programa principal (Main.java)

→Creación de objetos y pruebas de validación

- Instanciación de objetos
- Primer constructor de 6 parámetros
- Ejecución de un caso válido
- Prueba de la invariante de masa
- Manejo de excepciones con try-catch

```java
import java.util.Date;
// Flujo de ejecución
public class Main {
    Run | Debug
    public static void main(String[] args) {
        OperarioProduccion op = new OperarioProduccion(codigo: "OP-001", nombre: "Carlos Pérez", turno: "Matutino");
        GestorProduccion gestor = new GestorProduccion();

        try {
            // Ejemplo 1: Cumple con la masa y da un rendimiento de 48% (Bueno)
            RegistroFileteo reg1 = new RegistroFileteo(new Date(), lote: "LOTE-100", pesoInicial: 100.0, pesoLomos: 48.0, pesoDesperdicio: 52.0, op);
            gestor.agregarRegistro(reg1);
            System.out.println("Registro 1 agregado correctamente. Rendimiento: " + reg1.calcularRendimientoPorcentaje() + "% -> " + reg1.evaluarRendimiento());

            // Ejemplo 2: Rompe la regla de conservación de masa
            try {
                new RegistroFileteo (new Date (), lote: "LOTE-101", pesoInicial: 100.0, pesoLomos: 40.0, pesoDesperdicio: 50.0, op);
            } catch (IllegalArgumentException e) {
                System.out.println("Error detectado correctamente: " + e.getMessage());
            }
```

→Evaluación y cierre de producción

- Uso de constructor de 5 parámetros
- Registro de la producción
- Evaluación de rendimiento
- Cierre del turno
- Control de excepciones

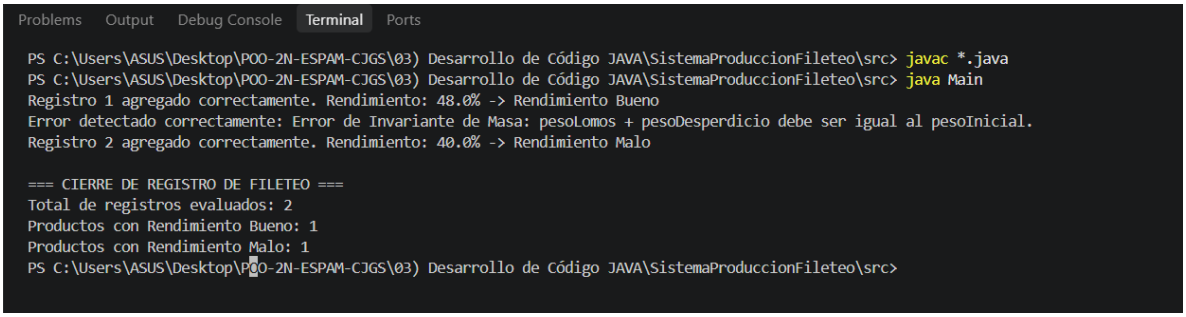
```java
            // Ejemplo 3: Cumple con la masa pero da un rendimiento de 40% (Malo)
            RegistroFileteo reg2 = new RegistroFileteo(lote: "LOTE-102", pesoInicial: 100.0, pesoLomos: 40.0, pesoDesperdicio: 60.0, op);
            gestor.agregarRegistro(reg2);
            System.out.println("Registro 2 agregado correctamente. Rendimiento: " + reg2.calcularRendimientoPorcentaje() + "% -> " + reg2.evaluarRendimiento());

            System.out.println();
            // Ejecutar el cierre de turno
            gestor.cerrarRegistroFileteo();

        } catch (IllegalArgumentException e) {
            System.err.println("Validación fallida: " + e.getMessage());
        }
    }
}
```

Ejecución correcta del sistema, validación de masa y clasificación del rendimiento.

- Funcionamiento general del programa
- Validación de la conservación de masa
- Clasificación de los rendimientos como buenos o malos
- Generación del resumen de producción



3.4. Desarrollo de código en Python

Justificación

Para migrar el sistema de gestión de producción y fileteo de atún se escogió Python, porque este lenguaje posee una sintaxis sencilla, clara y de fácil comprensión. Además, permite implementar principios de POO como el encapsulamiento, constructores, métodos y validaciones.

La migración tiene como objetivo comprobar que la lógica del sistema puede mantenerse en otro lenguaje de programación conservando las reglas del proceso y el comportamiento esperado, aunque se modifique la manera de escribir el código.

Equivalencias de sintaxis entre Java y Python

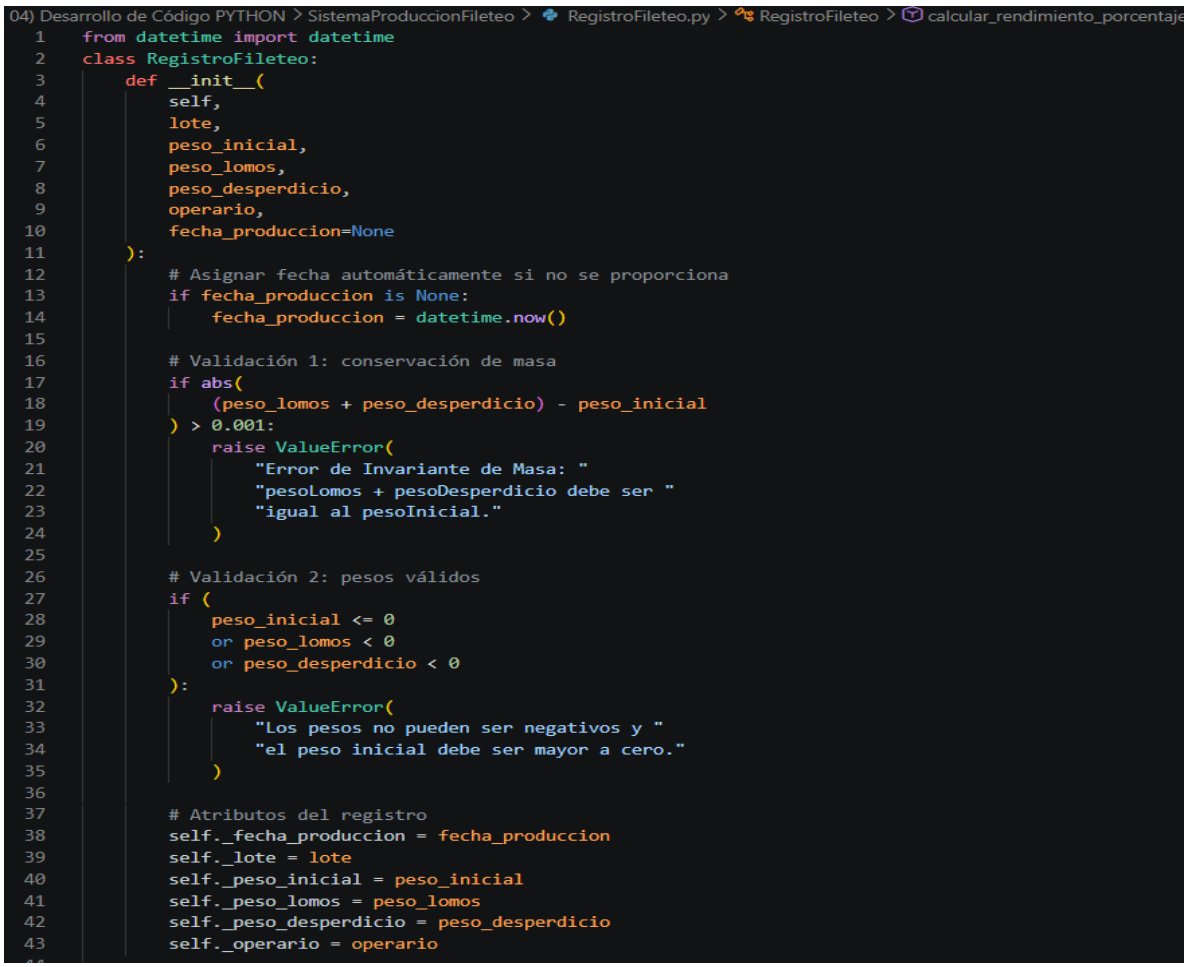
Concepto	Java	Python
Clase	public_class	Class
Atributos privados	private	_atributo (convención)
constructores	nombre de la clase	_init_()

Lanzar un error	throw new IllegalArgumentException()	raise ValueError()
Referencia al objeto	this	self
Condicional	if / else	If / else
Método	Tipo de retorno y nombre	Def y nombre del método
Excepciones	try / catch	try / except

Desarrollo de clase RegistroFileteo.py

→Definición de clase y constructor

- Invariante de conservación de masa
- Validación de pesos
- Encapsulamiento



→Métodos y getters

- Cálculo de porcentaje de rendimiento
- Evaluación de rendimiento
- Getters

```
45      # Calcular el porcentaje de rendimiento
46      def calcular_rendimiento_porcentaje(self):
47          return (self._peso_lomos / self._peso_inicial) * 100.0
48
49      # Evaluar el rendimiento
50      def evaluar_rendimiento(self):
51          rendimiento = self.calcular_rendimiento_porcentaje()
52
53          if 45.0 <= rendimiento <= 51.0:
54              return "Rendimiento Bueno"
55          else:
56              return "Rendimiento Malo"
57
58      # Getters
59      def get_fecha_produccion(self):
60          return self._fecha_produccion
61
62      def get_lote(self):
63          return self._lote
64
65      def get_operario(self):
66          return self._operario
67
68      def get_peso_lomos(self):
69          return self._peso_lomos
70
71      def get_peso_desperdicio(self):
72          return self._peso_desperdicio
73
74      def get_peso_inicial(self):
75          return self._peso_inicial
```

Implementación del programa principal (Main.py)

→Instanciación, registro y manejo de excepciones

- Importación de clases
- Creación de objetos
- Primer registro valido
- Registro invalido y control de excepción

04) Desarrollo de Código PYTHON > SistemaProduccionFileteo > Main.py > main

```python
from datetime import datetime

from OperarioProduccion import OperarioProduccion
from RegistroFileteo import RegistroFileteo
from GestorProduccion import GestorProduccion


def main():
    op = OperarioProduccion(
        "OP-001",
        "Carlos Pérez",
        "Matutino"
    )

    gestor = GestorProduccion()

    try:
        # Ejemplo 1: masa correcta y rendimiento bueno
        reg1 = RegistroFileteo(
            "LOTE-100",
            100.0,
            48.0,
            52.0,
            op,
            fecha_produccion=datetime.now()
        )

        gestor.agregar_registro(reg1)

        print(
            "Registro 1 agregado correctamente. "
            f"Rendimiento: "
            f"{reg1.calcular_rendimiento_porcentaje()}% "
            f"-> {reg1.evaluar_rendimiento()}"
        )
```

```
        # Ejemplo 2: incumple la conservación de masa
        try:
            RegistroFileteo(
                "LOTE-101",
                100.0,
                40.0,
                50.0,
                op,
                fecha_produccion=datetime.now()
            )
        except ValueError as error:
            print(
                "Error detectado correctamente: "
                f"{error}"
            )

```

→Evaluación y cierre de producción

- Creación del segundo registro
- Registro válido rendimiento fuera del rango establecido
- Evaluación de rendimiento
- Registro de lote en el gestor de producción
- Ejecución de cierre de turno
- Control de excepciones ante posibles errores.

```
        # Ejemplo 3: masa correcta y rendimiento malo
        reg2 = RegistroFileteo(
            "LOTE-102",
            100.0,
            40.0,
            60.0,
            op
        )

        gestor.agregar_registro(reg2)

        print(
            "Registro 2 agregado correctamente. "
            f"Rendimiento: "
            f"{reg2.calcular_rendimiento_porcentaje()}% "
            f"-> {reg2.evaluar_rendimiento()}"
        )

        print()

        # Ejecutar el cierre de turno
        gestor.cerrar_registro_fileteo()

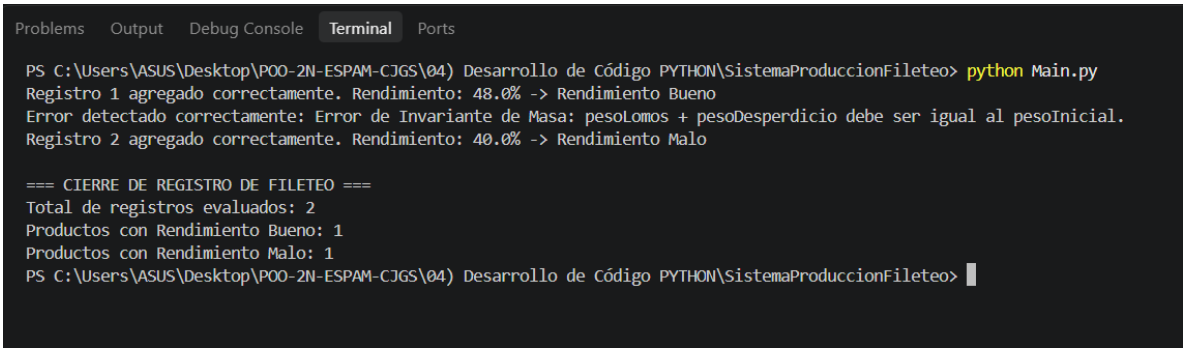
    except ValueError as error:
        print(f"Validación fallida: {error}")


if __name__ == "__main__":
    main()

```

Ejecución Python:

La ejecución permite comprobar que el sistema desarrollado en Python realiza correctamente las validaciones, maneja los errores y muestra los resultados del proceso de producción, conservando la misma lógica y funcionamiento que la versión implementada en Java.



4. Conclusión

El desarrollo del sistema de gestión de producción y fileteo de atún permitió poner en práctica los fundamentos de la Programación Orientada a Objetos definiendo clases, atributos, métodos y relaciones que representan las etapas del proceso productivo. Además, el encapsulamiento y las invariantes ayudaron a proteger los datos y garantizar que se cumplieran las reglas establecidas.

Asimismo, la migración de Java a Python permitió comprobar que un sistema puede conservar su lógica y funcionamiento al implementarse en diferentes lenguajes de programación. Aunque cada lenguaje presenta una sintaxis particular, se mantuvieron las validaciones y las reglas de negocio, obteniendo resultados equivalentes durante las pruebas realizadas.

Finalmente, esta experiencia permitió reconocer que aplicar correctamente los principios de la Programación Orientada a Objetos facilita el desarrollo de sistemas organizados, confiables y fáciles de mantener. También demuestra que un buen diseño permite adaptar una solución a distintos lenguajes sin perder su funcionamiento principal.